Emerging Directions in Probabilistic Modeling

Clark Miyamoto (cm6627@nyu.edu)

October 5, 2026

Contents

1 Arthur Gretton: Gradient Flows on MMD **2**
1.1 Sobolev MMD Flow 3

2 David Alvarez-Melis: Optimal Transport for Reasoning about Distribution **4**

3 **5**

4 Claire Boyer: Theoretical Insights into how Attention Learns Structure **7**

5 Yongxin Chen: Efficient Reinforcement Learning for Diffusion Models **9**

6 Sherry Yang: Reinforcement Learning in High-Cost Environments **10**

7 Omar Chehab: Flow-based density evaluation in one NFE **11**

1 Arthur Gretton: Gradient Flows on MMD

Definition 1 (Maximum Mena Discrepancy)**.** Let $\mathcal{H}$ be a Hilbert space

$$\sup_{\|f\|\leq 1} [\mathbb{E}_\nu f(X) - \mathbb{E}_{\nu^*} f(Y)] \tag{1.1}$$

where $f(x) = \langle f, \varphi(x)\rangle_{\mathcal{H}}$. Which implicitly defines a kernel $\langle \varphi(x), \varphi(x')\rangle_{\mathcal{H}} := k(x, x')$

This has an equivalent closed form expression

$$\mathsf{MMD}^2(\nu, \nu^*) = \mathbb{E}_\nu k(x, x') + \mathbb{E}_{\nu^*} k(y, y') - 2\mathbb{E}_{\nu,\nu^*} k(x, y) \tag{1.2}$$

Definition 2 (Wittness Function)**.**

$$f^*(x) \propto \mathbb{E}_\nu[k(X, x)] - \mathbb{E}_{\nu^*}[k(Y, x)] \tag{1.3}$$

Application: NN Training Dynamics Consider a probablity measure on neural network parameters

$$\nu_N = \frac{1}{N}\sum_{i=1}^N \delta_{Z_i} \tag{1.4}$$

You want to understand the curve ν_N^t describe by gradient descent

$$Z_i^{t+1} = Z_i^t - \gamma \nabla \mathcal{F}(\nu_N^t), \quad \mathcal{F} = \mathsf{MSE}(\nu_N^t; \mathcal{D}) \tag{1.5}$$

In the infinite width limit, the average over the empirical measure because an average over a full measure.

$$\mathcal{F}(\nu) \overset{N\to\infty}{\to} \mathbb{E}_{x,y}\|y - \mathbb{E}_{Z\sim\nu}\phi_Z(x)\|^2 \tag{1.6}$$

Assumption 1. Assert that $\nu, \nu^* \in \mathcal{P}_2(\mathbb{R}^d)$

Assume the well specified setting, that $y(x) = \mathbb{E}_{U\sim\nu^*}[\phi_U(x)]$, so the loss function becomes equivalent to MMD

$$\mathcal{F} = \frac{1}{2}\mathsf{MMD}^2(\nu, \nu^*) \tag{1.7}$$

$$= \frac{1}{2\mathbb{E}_\nu} k(x, x') + \frac{1}{2}\mathbb{E}_{\nu^*} k(y, y') - \mathbb{E}_{\nu,\nu^*} k(x, y) \tag{1.8}$$

The first & 2nd term become self interaction terms, and the 3rd term is an interaction between particles of a different type.

Wasserstein Gradient Flow In *Log Concave Sampling* by Chewi, we can derive the Wasserstein gradient flow, which gives a continuity equation

$$\mathcal{F}((\mathsf{Id} + \epsilon h)_\# \nu) = \mathcal{F}(\nu) + \epsilon\langle \nabla_{W_2}\mathcal{F}(\nu), h\rangle_{L^2(\nu)} + O(\epsilon) \tag{1.9}$$

$$\implies \partial_t \nu_t = \nabla \cdot (\nu_t \nabla_{W_2}\mathcal{F}(\nu_t)) \tag{1.10}$$

Which is equivalent to the derivative to the first variation of $\mathcal{F}$.

Fact. First variation $\mathcal{F}'$ of $\mathcal{F} = \frac{1}{2}\mathsf{MMD}^2$, this is the witness function!

Back to NN training dynamics in continuous time So the continuous time population dynamics of the neurons becomes a gradient descent on the witness function

Assumption 2. Assume ν^* satisfies the Poincare inequality.

Recall Poincare inequality tells us continuous time Langevin converges in KL.

Assumption 3. Suppose $\exists q_t \in L^2(\nu^*)$ such that

$$\frac{d\nu_t}{d\nu^*} - 1 = T^r_{\nu^*} q_t, \quad \|q_t\|_{L^2(\nu^*)} < Q \tag{1.11}$$

The authors were not able to prove this condition holds for all time for all problems...

Definition 3.

$$\mathsf{DrMMD}^2(\nu, \nu^*) = (1+\lambda) \tag{1.12}$$

where $\lim_{\lambda\to 0} \mathsf{DrMMD}^2 = \chi^2$ and $\lim_{\lambda\to\infty} \mathsf{DrMMD}^2 = \mathsf{MMD}^2$.

This gives you a near-global converge of the DrMMD flow

Discretization Error You can do this...

1.1 Sobolev MMD Flow

Assumption 4. The same α, Q for all time t

$$m_{\nu_t} - m_{\nu^*} = S^{\alpha/2}_{\nu_t} q_t \tag{1.13}$$

Results The DrMMD flow converges much faster than the MMD flow.

Appendix MMD is not displacement convex but is mixture convex.

$$\mathcal{F}(t\nu + (1-t)\mu) \le t\mathcal{F}(\nu) + (1-t)\mathcal{F}(\mu) \tag{1.14}$$

This corresponds to a "mass teleportation" convexity. Think more about this...

2 David Alvarez-Melis: Optimal Transport for Reasoning about Distribution

Three tasks this talk will attempt to handle

1. Which training distribution is closest to the target distribution?
2. Can we construct a distribution which is a natural "interpolation" between two distributions?
3. Can we predict/infer/optimize with models which takes distributions as inputs.

Transfer learning Typically, we can use OT to determine if two distributions μ, ν are similiar or not (in Wasserstein distance). Typically if μ, ν are empirical measures, you can use Sinkhorn algorithm to compute this; but what if μ and ν are datasets with different labels (e.g. MNIST vs FashionMNIST)

Definition 4.

$$d(y, y') = W_2(p(x|y = y), p(x'|y = y')) \tag{2.1}$$

Distribution DEQs Your object is to find θ such that $\mu_\rho^* = F_\theta(\mu_\theta^*, \rho)$. Something you can do is to write it as the loss function

$$\frac{1}{2}\mathsf{MMD}^2(\mu, F_\theta(\mu, \rho)) \tag{2.2}$$

3

Optimization Consider the proximal form of gradient descent

$$x_{k+1} = x_k - \tau \nabla f(x_k) \tag{3.1}$$

$$= \arg\min_x \frac{1}{2}\|x - x_k\|^2 + \tau \langle \nabla f(x_k), x - x_k \rangle \tag{3.2}$$

Definition 5 (Bregman Divergence)**.** Let $\phi : \mathbb{R}^d \to \mathbb{R}$ be convex

$$D_\phi(x, y) = \phi(x) - \phi(y) - \langle \nabla \phi(y), x - y \rangle \tag{3.3}$$

This defines the *Mirror Descent algorithm*

$$x_{k+1} = \arg\min_x D_\phi(x, x_k) + \tau \langle \nabla f(x_k), x - x_k \rangle \tag{3.4}$$

$$= \nabla \phi^*(\nabla \phi(x_k) - f \nabla f(x_k)) \tag{3.5}$$

where ϕ^* is the convex conjugate of ϕ.

Definition 6 (DC)**.** f is DC (difference-of-convex) function if there exists f^-, f^+ such that

$$f = f^+ - f^- \tag{3.6}$$

There is an algorithm to minimize DC functions. This is called the Convex-Concave CCP algorithm.

Wasserstein Geometry More or less the same as prior talks...

Gradient Descent in Wasserstein Space Consider the Mirror Descent algorithm but in probability space

$$T_{k+1} = \arg\min_{T \in L^2(\mu_k)} \frac{1}{2}\|T - \mathsf{Id}\|^2_{L^2(\mu_k)} + \tau \langle \nabla_{W_2} \mathcal{F}(\mu_k), T - \mathsf{Id} \rangle_{L^2(\mu_k)} \tag{3.7}$$

$$\mu_{k+1} = (T_{k+1})_\# \mu_k \tag{3.8}$$

Convexity

Definition 7. Let $\alpha \geq 0$, teh functional is α-totally convex if for all $\mu \in \mathcal{P}_2(\mathbb{R}^d)$ if

$$D_\mu(T, S) \geq \frac{\alpha}{2}\|T - S\|^2_{L^2(\mu)} \tag{3.9}$$

where D is the Bregmann divergence

Example. Functionals which aren't convex (in general)

- W_2^2 is convex only along one curve
- KL when reference measure is non-convex
- MMD

So instead let's generalize our notion of DC functions, into DC functionals. Consider the decomposition

$$\mathcal{F}(\mu) = \frac{1}{2}\mathsf{MMD}^2(\mu, \nu) = \mathcal{V}_k(\mu) + \mathcal{W}_k(\mu) + \mathsf{const} \tag{3.10}$$

where $\mathcal{V}_k(\mu) = \mathbb{E}[V_k]$ and

Assumption 5. Let's assume we only focus on kernel that are only translation invariant $k(x, y) = \psi(x - y)$

So now we have the DC compositon $\mathcal{F} = \mathcal{F}^+ - \mathcal{F}^-$

4 Claire Boyer: Theoretical Insights into how Attention Learns Structure

Single-Location Regression Research Question: Why does attention work well for NLP tasks?

To answer this question, they wanted to design a toy-dataset that models token-wise sparsity and randomness in the position of relevant information. All the while, keeping the inference "linear" (in what space?)

Definition 8 (Single Location Regression)**.** Consider L random tokens $(X_1, ..., X_L) \in (\mathbb{R}^d)^L$. The output $Y \in \mathbb{R}$ is given by

$$Y = X_{J_0}^T v^* + \xi \tag{4.1}$$

where J_0 is a random index for X. To be specific it is a discrete random variable on $[L]$ and conditionally on J_0

$$X_{J_0} \sim \mathcal{N}\left(\sqrt{d/2}k^*, \gamma^2 \mathbb{I}_d\right) \tag{4.2}$$

$$X_\ell \sim \mathcal{N}(0, \mathbb{I}_d), \quad \ell \neq J_0 \tag{4.3}$$

meaning $\xi \sim \mathcal{N}(0, \epsilon^2)$.

Notationally, $(\mathbb{X}, Y) \sim \mathsf{SLR}(k^*, v^*)$. We want to find k^*, v^*.

Consider a single head attention operation for a prompt $\mathbb{X} = (X_1, ..., X_L)$

$$T_\lambda^{(K,Q,V)}[\mathbb{X}](x_q) = \frac{\sum_\ell \exp\left(\lambda \langle Q x_q, K X_\ell \rangle\right) V X_\ell}{\sum_{\ell'} \exp\left(\lambda \langle Q x_q, K X_{\ell'} \rangle\right)} \tag{4.4}$$

For example consider hte output of the first token [CLS] this token can be hidden inside the temperature λ. You'll notice this makes the optimization of λ, Q, K, V more difficult. So instead they replace the softmax by a component-wise non-linearity

$$T_\lambda^{(K,V)} = \mathsf{erf}(...) \tag{4.5}$$

They now declare the dynamics on the riskk function

$$\mathcal{R}_\lambda(k, v) = \mathbb{E}_{\mathbb{X}, Y \sim \mathsf{SLR}(k^*, v^*)} \left[\left(Y - T_\lambda^{(k,v)}[\mathbb{X}] \right)^2 \right] \tag{4.6}$$

They find

Fact (Informal)**.**

- With proper initialization & step size, projected gradient descent on $\mathcal{R}_\lambda$ satisfies

$$(k_t, v_t) \overset{t \to \infty}{\to} \pm(k^*, v^*) \tag{4.7}$$

- As $d \to \infty$, the predictor $T_\lambda^{(k^*, v^*)}$ is asymptotically Bayes optimal.

Measure-Based View To make a nicer claim without using the erf approximation, they made a measure-based connection to the attention operator.

$$T^{K,Q,V} = \frac{\sum_\ell \exp(z_\ell^T K^T Q z_q) V z_\ell}{\sum_{\ell'} \exp(z_\ell^T K^T Q z_q)} \to \frac{\int \exp(\tilde{z}^T K^T Q z) V \tilde{z} \, d\mu(\tilde{z})}{\int \exp(\tilde{z}^T K^T Q z) \, d\mu(\tilde{z})} \tag{4.8}$$

Fact. Assume the input measure is Gaussian. $\mu = \mathcal{N}(m, \Sigma)$. Then the large-limit attention operator if just an affine map

$$T^{K,Q,V} = Vm + V\Sigma K^T Qz \tag{4.9}$$

Fact (Informal, Concetration)**.** If μ and ν are σ-sub-Gaussian measures, then there exists constants c_1, c_2 depending on d, V, K, Q such that

$$\mathbb{E}\|T[\hat{\mu}_L] - T[\mu]\|^2_{L^2(\mu)} \leq c_1\sigma^6 \frac{\log L}{L^{c_2/\sigma^2}} \tag{4.10}$$

Under additional mild assumption, the gradient also concentrates

$$\mathbb{E}\|\nabla T[\hat{\mu}] - \nabla T[\mu]\|^2_{L^2(\nu)} \leq c_1\sigma^{12} \frac{(\log L)^2}{L^{c_2/\sigma^2}} \tag{4.11}$$

Training Dynamics Now, the goal is to do the training dynamics analysis in $L \to \infty$, and then bring it back down to finite L.

5 Yongxin Chen: Efficient Reinforcement Learning for Diffusion Models

In DPO, your objective is to maximize the reward R while staying close to a pretrained model π_{ref}

$$\max_{\theta} \mathbb{E}_{c\sim\mathcal{D}}\mathbb{E}_{x\sim\pi_\theta(\cdot|c)}[R(x,c)] - \beta\mathbb{E}_{c\sim\mathcal{D}}\mathsf{KL}(\pi_\theta(\cdot|c)\|\pi_{ref}(\cdot|c)) \tag{5.1}$$

For autoregressive LLMs the model factorizes

$$\pi_\theta(y|c) = \prod_k \pi_\theta(y_k|y_{<k},c) \tag{5.2}$$

In practice how do you run this optimization? You can do GRPO

$$\mathcal{L}_{GRPO} = -\mathbb{E}\left[\sum_{i,k} \min\left(\rho_{i,k}A_i,\ \mathsf{clip}(\rho_{i,k},1-\epsilon,1+\epsilon)A_i\right)\right] + \beta\mathsf{KL}(\pi_\theta\|\pi_{ref}) \tag{5.3}$$

$$\rho_{i,k} = \frac{\pi_\theta(y_{i,k}|y_{i,<k},c)}{\pi_{old}(y_{i,k}|y_{i,<k},c)} \tag{5.4}$$

$$A_i = \frac{R(y_i,c) - \mathsf{mean}_j R(y_j,c)}{\mathsf{std}_j R(y_j,c)} \tag{5.5}$$

Or a policy gradient

$$\mathcal{L}_{PG} = -\mathbb{E}\left[\sum_{ik} A_i \mathsf{sg}(\rho_{i,k}) \log \pi_\theta(y_{i,k}|y_{i,<k},c)\right] + \beta\mathsf{KL}(\pi_\theta\|\pi_{ref}) \tag{5.6}$$

In diffusion, the token ratio $\rho_{i,k}$ can be taken to be the denoising transition ratio (that is do Euler-Maruyama, and write the conditional proposal induced by that discretization). This is now a ratio of Gaussians, so this can be evaluated analytically

Reward-Based Velocity Matching However in flows, you only have access to v_θ, not π_θ. So it'd be nice to have a objective which is a function of v_θ.

$$\mathcal{L}_{RVM} = \mathbb{E}[r(x_0)\|v_\theta - v\|^2 + \beta(r(x_0))\|v_\theta - v_{anc}\|^2]i \tag{5.7}$$

where v is the target velocity (in a stochastic interpolants language, this is $v = \dot{I}_t$).

1. RAM: v_{anc} is a frozen reference
2. v_{anc} is a lagged EMA of v_θ
3. ... Missed this, go back to paper https://arxiv.org/pdf/2608.23664

This doesn't correspond to a proper tilt $p \propto \pi \times e^{\beta r(x)}$, but rather it's just supposed to get you sample that evaluate with high reward.

6 Sherry Yang: Reinforcement Learning in High-Cost Environments

Research question: How can we run RL when the environment is computationally/time-wise expensive (e.g. recursive self improvement, engineering, etc.)

Reward-Rate Policy Gradients ...

MLE-World ...

7 Omar Chehab: Flow-based density evaluation in one NFE

Goal: Can we use a neural network to learn an probability density function. Previously people use normalizing flows, but we'd like to do this on continuous normalizing flows.

> **Example** (Boltzmann Generator in Chemistry). Learn p_{model} from data. Get un-normalie $\tilde{p}_{target}(x) = \exp(-E(x))$. Do importance sampling; sample p_{target} by reweighting samples from p_{model}
>
> $$p_{target}(\cdot) \approx \sum_i \frac{\frac{\tilde{p}_{target}}{p_{model}}}{\sum_j \frac{\tilde{p}_{target}(x_j)}{p_{model}(x_j)}} \tag{7.1}$$

Probability using Flows In a flow, your neural network parameterizes $v^\theta \approx v$. Recall you can find the density by running

$$\log p_1(X_1) = \log p_0(X_0) + \int_0^1 -\nabla \cdot v_t(X_t)\, dt \tag{7.2}$$

where $X_1 = X_0 + \int_0^1 v_t(X_t)\, dt$. Notice you have to compute an integral; so this could be potentially very expensive.

In practice, people we do this by using Hutchinson's estimator

$$\nabla \cdot v_t(X_t) = \mathrm{Tr}(\nabla v_t(X_t)) = \mathbb{E}_{\epsilon \sim \mathcal{N}(0,\mathbb{I})}[\epsilon^T \nabla v_t(X_t)\epsilon] \approx \epsilon^T \nabla v_t(X_t)\epsilon \tag{7.3}$$

which is unbiased but has high variance. Where $\nabla v \in \mathbb{R}^{d\times d}$ denotes the Jacobian.

A dumb idea to solve this is to use a neural network to approximate this

$$\mathcal{L}(\theta) = \mathbb{E}\|D_\theta(x_t, t, t) - (-\nabla \cdot v_t(x_t))\|^2 \tag{7.4}$$

$$= \mathbb{E}\|D_\theta(x_t, t, t) - \frac{d}{dt}\log p_t(x_t)\|^2 \tag{7.5}$$

$$= \mathbb{E}\|D_\theta(x_t, t, t) - (\partial_t \log p_t(x_t) + v_t(x_t)^T \nabla \log p_t(x_t))\|^2 \tag{7.6}$$

See https://arxiv.org/pdf/2606.29110 for more...

Spatial Temporal Methods for Evaluating Densities ...

buffer